ENTERPRISE PRODUCTION ARCHITECTURE & OPERATIONS

WireGuard Enterprise VPN Gateway

Zero-Trust High-Throughput Linux In-Tree Gateway & FastAPI Control Plane

Host Server: 10.31.7.48

VPN Subnet: 172.29.250.0/24 (Gateway: 172.29.250.1)

Listen Port: UDP 443 (Firewall Bypass)

Version: v4.2 ZTNA Production

1. Executive Summary & Design Principles

WireGuard Enterprise couples the line-rate throughput and cryptographic simplicity of the Linux in-tree kernel module (`wireguard.ko`) with a fully containerized Zero-Trust Network Architecture (ZTNA) control plane. Designed for high-assurance, air-gapped, and mission-critical enterprise environments, all data plane packets stay strictly within kernel space, achieving sub-millisecond latencies and eliminating user-space copying overheads.

In-Tree Kernel Data Plane

Line-rate encryption via ChaCha20-Poly1305 and Curve25519 ECDH. Direct packet forwarding without user-space buffer bottlenecks.

Zero-Trust Firewall Pipeline

Granular per-client (/32) NFTables policy verification. Client-to-client isolation with dedicated ICMP/SSH management tunnels.

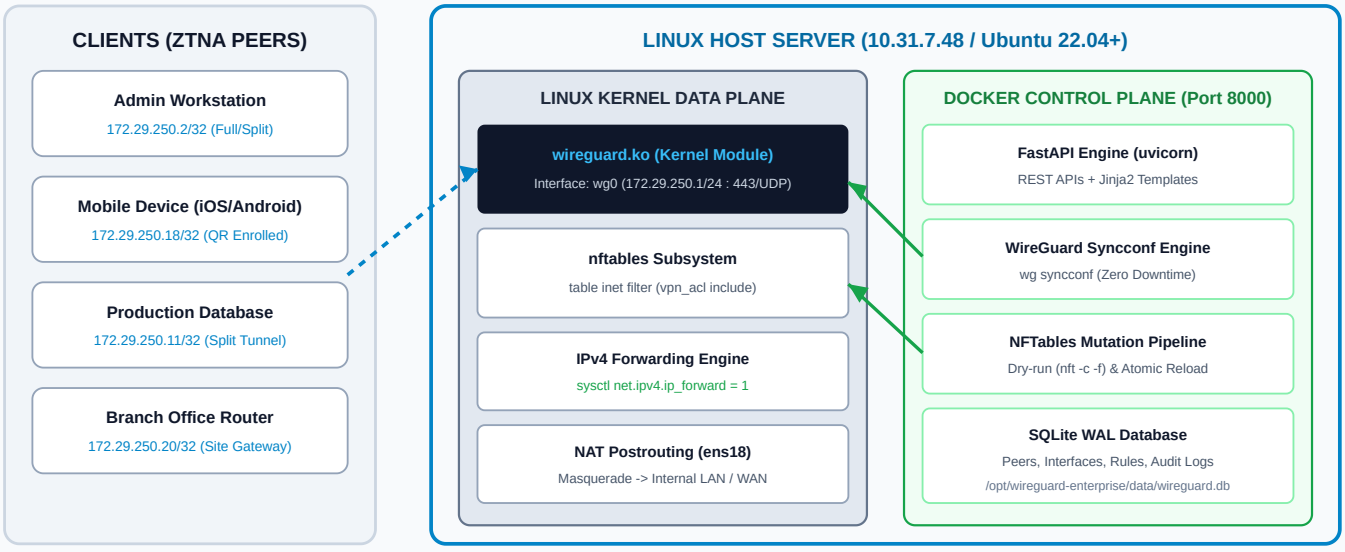
Air-Gapped Independence

Zero external CDN or cloud dependencies. 100% self-hosted vector UI, local QR code generation, and single-tarball deployment.

2. High-Level System Architecture

The architecture enforces strict decoupling between the **Control Plane** (FastAPI Docker container) and the **Data Plane** (Kernel WireGuard & NFTables).

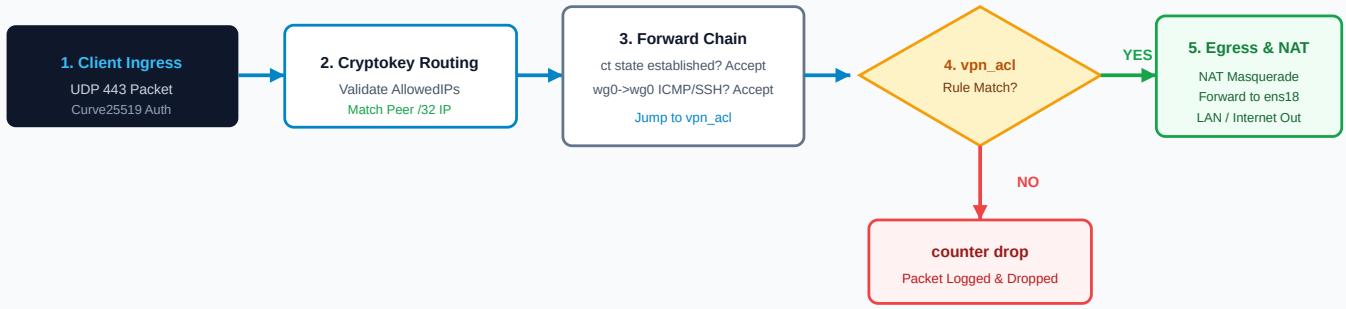
FIGURE 1: END-TO-END SYSTEM & NETWORK ARCHITECTURE



3. End-to-End Client Packet Flow & Routing

When a client sends traffic through the tunnel, packets undergo cryptographic authentication inside kernel space before being subjected to stateful inspection and Zero-Trust ACL evaluation.

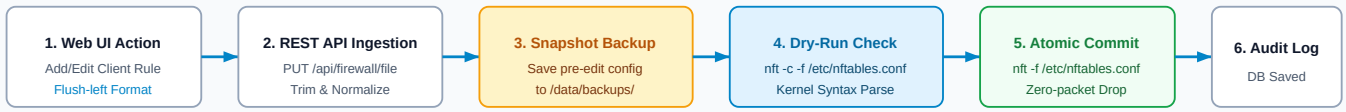
FIGURE 2: INGRESS PACKET STATEFUL INSPECTION & ROUTING LIFECYCLE



4. Zero-Trust Access Control (ZTNA) & Mutation Pipeline

Security policies are expressed in `/etc/nftables/acl.nft`, which is included inside `chain vpn_acl`. All edits from the web dashboard pass through an atomic 6-stage validation pipeline:

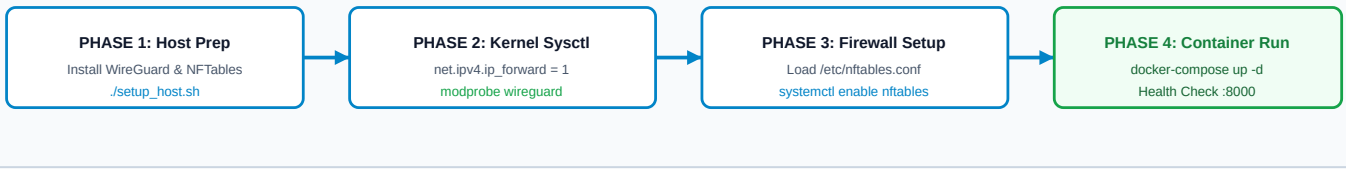
FIGURE 3: SAFE FIREWALL MUTATION & KERNEL COMMIT FLOWCHART



5. Complete Server Deployment Lifecycle

Deploying a new WireGuard Enterprise server follows a strict 4-phase sequential workflow:

FIGURE 4: AUTOMATED DEPLOYMENT LIFECYCLE FLOWCHART



6. Step-by-Step Server Deployment Commands

Step 1: Host System Preparation

```
# 1. Update OS package repositories
sudo apt-get update -y && sudo apt-get install -y wireguard wireguard-tools nftables iptables-persistent docker.io
docker-compose curl jq

# 2. Enable persistent IPv4 forwarding and BBR congestion control
sudo tee /etc/sysctl.d/99-wireguard-enterprise.conf << 'EOF'
net.ipv4.ip_forward = 1
net.ipv4.conf.all.forwarding = 1
net.ipv4.tcp_syncookies = 1
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
EOF
sudo sysctl --system

# 3. Load WireGuard kernel module
sudo modprobe wireguard
echo "wireguard" | sudo tee -a /etc/modules-load.d/wireguard.conf
```

Step 2: Directory Structure & File Permissions

```
sudo mkdir -p /etc/wireguard /etc/wireguard/archive /etc/nftables /opt/wireguard-enterprise/app /opt/wireguard-ente
rprise/data
sudo chmod 700 /etc/wireguard
sudo chmod 755 /etc/nftables
sudo chmod 750 /opt/wireguard-enterprise
```

Step 3: Baseline NFTables Configuration

```
# Create baseline /etc/nftables/acl.nft
sudo touch /etc/nftables/acl.nft && sudo chmod 644 /etc/nftables/acl.nft

# Copy template and atomic reload
sudo cp server_deployment/nftables.conf.template /etc/nftables.conf
sudo nft -c -f /etc/nftables.conf   # Syntax validation
sudo nft -f /etc/nftables.conf      # Live atomic reload
sudo systemctl enable --now nftables.service
```

Step 4: Launching Application Container

```
# Sync codebase to host location
sudo cp -ru app/ /opt/wireguard-enterprise/

# Start container using Docker Compose with host networking
cd server_deployment
sudo docker-compose up -d --force-recreate

# Verify container health
sudo docker ps -f name=wireguard-manager
curl -s http://127.0.0.1:8000/login | grep "WireGuard"
```

7. Network Topology & Ports Reference

Interface	Network / IP	Protocol & Port	Purpose	Access Policy
ens18	10.31.7.48/24	TCP 8000	Web Admin Dashboard	Restricted to Management Subnet
ens18	10.31.7.48/24	TCP 22	Host SSH Management	Restricted Admin Jump Host
ens18	10.31.7.48/24	UDP 443	WireGuard Encrypted Tunnel	Public / Corporate WAN Allowed
wg0	172.29.250.1/24	All Protocols	VPN Internal Gateway	Internal Virtual Interface
wg0 (Peers)	172.29.250.2-254/32	ICMP / TCP 22	Peer Management & Diagnostics	Client-to-Client Allowed
wg0 (Peers)	172.29.250.2-254/32	Dynamic Ports	LAN / Target Server Access	Evaluated via `/etc/nftables/acl.nft`

8. Operational & Maintenance Cheatsheet

1. Container & Process Operations

```
# Check container live logs
docker logs -f --tail 50 wireguard-manager

# Restart application container with zero kernel disruption
sudo docker restart wireguard-manager

# Execute an interactive shell inside the container
docker exec -it wireguard-manager bash
```

2. WireGuard Kernel State Inspection

```
# Inspect live handshakes, endpoints, and Rx/Tx counters
sudo wg show wg0

# Check virtual network interface details
ip -s addr show wg0
ip route show dev wg0

# Re-sync server config without tearing down tunnel
sudo wg syncconf wg0 <(sudo wg-quick strip /etc/wireguard/wg0.conf)
```

3. Firewall & ACL Management

```
# List live rules in the active kernel table
sudo nft list ruleset

# Specifically view client ACL chain rules
sudo nft list chain inet filter vpn_acl

# Manually validate and reload firewall rules
sudo nft -c -f /etc/nftables.conf && sudo nft -f /etc/nftables.conf
```

4. Backup & Disaster Recovery

```
# 1. Immediate SQLite Database Backup
sudo cp /opt/wireguard-enterprise/data/wireguard.db /opt/wireguard-enterprise/data/wireguard_backup_$(date +%Y%m%d%H%M%S).db

# 2. Archive Current WireGuard Configuration
sudo cp /etc/wireguard/wg0.conf /etc/wireguard/archive/wg0_backup_$(date +%Y%m%d%H%M%S).conf

# 3. Archive Firewall Configuration
sudo cp /etc/nftables/acl.nft /etc/wireguard/archive/acl_backup_$(date +%Y%m%d%H%M%S).nft
```

Production Support & Compliance Note

All cryptographic peer identities are stored in `/opt/wireguard-enterprise/data/wireguard.db` with client private keys encrypted using PBKDF2 AES-GCM primitives. The in-tree WireGuard data plane preserves active connections across container restarts. For troubleshooting, inspect `dmesg | grep wireguard` or contact the infrastructure administrator.